

# ALP

FICHA DE EXERCÍCIOS  
ATIVIDADE LETIVA

Programação Orientada a Objetos

UNIDADE CURRICULAR

Ficha 5 – Renderização Dinâmica com Template Strings

FICHA

Crie uma pasta para esta ficha (sugestão de nome: F05\_Soundbase Render).

Dentro dessa pasta, coloque os ficheiros disponíveis no Moodle: `index.html`, `style.css`, `soundbase-dados.js` e `soundbase-classes.js`.

Crie também o ficheiro `script.js` e adicione-o ao `index.html` como último `<script>`.

Todo o código JavaScript deve estar no ficheiro `script.js`.

Utilize `console.log()` para verificar resultados intermédios.

Esta ficha introduz renderização dinâmica com template strings. Os exercícios são progressivos e baseiam-se no mesmo cenário: o SoundBase. O objetivo é construir diferentes componentes de interface - cards, tabela e modal - a partir de instâncias das classes `Track` e `Playlist`. Leia cada exercício na totalidade antes de começar.

## Ficheiros disponíveis no Moodle

| Ficheiro                          | Conteúdo                                                                                                                     |
|-----------------------------------|------------------------------------------------------------------------------------------------------------------------------|
| <code>index.html</code>           | Estrutura base da página com todos os seletores necessários                                                                  |
| <code>style.css</code>            | Estilos visuais da aplicação                                                                                                 |
| <code>soundbase-dados.js</code>   | Array <code>catalogue</code> com objetos literais; funções auxiliares <code>formatDuration</code> e <code>formatPlays</code> |
| <code>soundbase-classes.js</code> | Classes <code>Track</code> e <code>Playlist</code> prontas a usar                                                            |

## Estrutura do `index.html`

| Seletor                       | Descrição                                                                          |
|-------------------------------|------------------------------------------------------------------------------------|
| <code>#app-title</code>       | Título principal da aplicação ( <code>&lt;h1&gt;</code> )                          |
| <code>#view-controls</code>   | Contentor com botões de vista (atributo <code>data-view: "cards"/"tabela"</code> ) |
| <code>#filter-controls</code> | Contentor com botões de filtro por género (atributo <code>data-genre</code> )      |

|                            |                                                                                                   |
|----------------------------|---------------------------------------------------------------------------------------------------|
| #catalogue-grid            | Contentor <div> para renderização em cards                                                        |
| #catalogue-table-container | Contentor <div> para renderização em tabela (inicialmente oculto com classe <code>hidden</code> ) |
| #modal-overlay             | Overlay do modal (inicialmente oculto com classe <code>hidden</code> )                            |
| #modal-content             | Contentor interno do modal onde o conteúdo é injetado.                                            |

### Preparação dos Dados

- 1) O array `catalogue` em `soundbase-dados.js` contém objetos literais. Use `Track.fromObject()` para converter cada elemento numa instância de `Track` e guarde o resultado num novo array chamado `tracks`.
  - Use `map()` para a conversão.
  - Confirme com `console.log(tracks[0].getInfo())` que o primeiro elemento é uma instância válida de `Track`.
- 2) Crie uma instância de `Playlist` chamada `myPlaylist` com o nome "My Playlist" e o utilizador "soundbase\_user". Não adicione faixas por agora.
  - Confirme com `console.log(myPlaylist.size)` que a playlist está vazia.

### Renderização como Cards

- 3) Crie uma função `renderCard(track)` que recebe uma instância de `Track` e retorna uma string HTML com a seguinte estrutura (os valores são um exemplo):

```
<div class="track-card" data-id="a1b2c3d4-...">
  <div class="card-header">
    <h3>Blinding Lights</h3>
    <span class="genre-tag">Pop</span>
  </div>
  <p class="artist">The Weeknd</p>
  <p class="meta">3:20 · 8.4k plays</p>
  <div class="card-actions">
    <button class="btn-play" data-id="a1b2c3d4-..." aria-label="Play Blinding Lights">▶ Play</button>
    <button class="btn-details" data-id="a1b2c3d4-..." aria-label="Details for Blinding Lights">Details</button>
  </div>
</div>
```

- 4) Crie uma função `renderCards(tracks)` que:
    - Recebe um array de instâncias de `Track`.
    - Use `map()` para transformar cada instância na sua representação HTML com `renderCard`.
    - Junta os resultados com `join("")` e atribui ao `innerHTML` do `#catalogue-grid`.
  - 5) Chame `renderCards(tracks)` para renderizar o catálogo completo. Confirme o resultado no browser.
- Nota:** A função de renderização (`renderCard`) produz HTML; a função de injeção (`renderCards`) coloca-o no DOM. Mantenha estas responsabilidades separadas – cada função deve ter uma única tarefa. As funções de renderização ficam fora das classes propositadamente: produzir HTML é uma responsabilidade de apresentação, não dos dados.

## Renderização como Tabela

- 6) Crie uma função `renderRow(track)` que recebe uma instância de `Track` e retorna uma string HTML com uma linha de tabela (`<tr>`):
  - As colunas devem ser, por esta ordem: título, artista, género, duração formatada, plays formatados.
  - Inclua o atributo `data-id` no `<tr>`.
- 7) Crie uma função `renderTable(tracks)` que:
  - Constrói uma string HTML completa com a estrutura `<table> → <thead> → <tbody>`.
  - O `<thead>` deve ter as colunas: Title, Artist, Genre, Duration, Plays.
  - O `<tbody>` deve ser preenchido com os resultados de `renderRow()` para cada faixa.
  - Atribui o resultado ao `innerHTML` do `#catalogue-table-container`.

Nota: Ao contrário dos cards, a tabela é construída como uma única string HTML – a estrutura completa, incluindo `<thead>`, é gerada dentro da função.

## Filtros e Re-renderização

- 8) Adicione um event listener ao `#filter-controls` que reage a cliques com delegação de eventos.
  - Verifique se o elemento clicado tem a classe `btn-filter`; se não, ignore o evento.
  - Leia o valor do atributo `data-genre` do botão clicado.
  - Filtre o array `tracks` com `filter()`: se o género for "all", mostre todas as faixas; caso contrário, mostre apenas as que correspondem ao género.
  - Chame `renderCards()` com o resultado filtrado.
- 9) Atualize o event listener anterior para destacar o botão ativo.

## Modal de Detalhes

- 10) Crie uma função `renderModal(track)` que retorna uma string HTML para o conteúdo do modal (os valores são um exemplo):
 

```
<div class="modal-card">
  <button class="btn-close" aria-label="Close modal">X</button>
  <h2>Blinding Lights</h2>
  <p>Artist: The Weeknd</p>
  <p>Genre: Pop</p>
  <p>Duration: 3:20</p>
  <p>Plays: 8.4k</p>
  <p>Liked: No</p>
</div>
```
- 11) Adicione um event listener ao `#catalogue-grid` para cliques com delegação de eventos.
  - Se o elemento clicado tiver a classe `btn-details`, leia o `dataset.id` do botão.
  - Encontre a faixa correspondente em `tracks` com `find()`, comparando o `id` da instância.
  - Injete o resultado de `renderModal(track)` no `innerHTML` de `#modal-content`.
  - Remova a classe `hidden` do `#modal-overlay`.
- 12) Implemente o encerramento do modal:
  - Adicione um event listener ao `#modal-overlay` para o evento `click`.

- Se o elemento clicado for o próprio overlay (`event.target === event.currentTarget`) ou tiver a classe `btn-close`, adicione a classe `hidden` ao `#modal-overlay`.

Nota: Verificar `event.target === event.currentTarget` garante que clicar dentro do `modal-card` não fecha o modal - apenas o overlay ou o botão `×` o fazem.

### Exercício Integrador

13) Implemente a troca de vistas com os botões de `#view-controls`.

- Declare uma variável `currentFilter` com o valor inicial `"all"`. Esta variável representa o estado atual do filtro e deve ser atualizada sempre que um botão de filtro é clicado.
- Ao clicar em "Cards": mostre `#catalogue-grid` e oculte `#catalogue-table-container` (use a classe `hidden`); chame `renderCards()` com as faixas correspondentes a `currentFilter`.
- Ao clicar em "Table": oculte `#catalogue-grid` e mostre `#catalogue-table-container`; chame `renderTable()` com as faixas correspondentes a `currentFilter`.
- Atualize também o event listener dos filtros para manter `currentFilter` sincronizado e re-renderizar a vista atualmente ativa.

### Desafios

14) Implemente ordenação das faixas:

- Adicione ao HTML um elemento `<select id="sort-controls">` com as opções "By title (A-Z)", "By plays (↓)" e "By duration (↓)".
- Ao alterar a seleção, ordene uma cópia do array filtrado e re-renderize a vista ativa. Não modifique o array `tracks` original.

15) Implemente "Add to Playlist":

- Adicione um botão `btn-add` a cada card em `renderCard`. Ao clicar, adicione a faixa à `myPlaylist` com o método `add()`.
- Crie uma função `renderPlaylist(playlist)` que injeta num elemento `#playlist-info` o nome da playlist, o número de faixas e a duração total; chame-a sempre que uma faixa for adicionada.